Chapter 1

RTL Design & Implementation of Digital Signal Processors

1.1 IP Block 7: Digital Differentiator (Calculus in Silicon)

In analog circuit design, performing a mathematical derivative on a voltage signal requires a physical capacitor and resistor network. The current through a capacitor is strictly proportional to the rate of change of the voltage across it ($I = C\frac{dV}{dt}$). In modern Mixed-Signal System-on-Chip (SoC) architectures, heavy analog components are highly undesirable. Instead, calculus must be performed in real-time using pure digital logic.

The **Digital Differentiator** IP block acts as a hardware-accelerated calculus engine. It ingests an 8-bit continuous analog waveform and computes its exact instantaneous rate of change, outputting the mathematical derivative.

1.1.1 Detailed Working Mechanism & Theory of Operation

To translate continuous analog calculus into discrete digital logic, the IP block utilizes the **First-Order Difference Equation**:

$$y[n] = x[n] - x[n-1] \tag{1.1}$$

Where $x[n]$ is the current 8-bit ADC sample, and $x[n-1]$ is the previous sample captured one clock cycle prior.

Handling Negative Slopes (DC Biasing)

A physical analog signal can have a negative slope (e.g., a falling wave). However, standard Data Acquisition (DAQ) systems and 8-bit DACs operate on unsigned integers (0 to 255), which cannot represent negative numbers. To prevent mathematical underflow, the digital hardware applies a strict **DC Offset of 128** (2.5V mid-rail). A slope of zero (flat line) outputs exactly 128. Positive slopes step above 128, and negative slopes step proportionally below 128.

The Integer Quantization Trap & Digital Gain

During initial architectural testing, a severe quantization loss anomaly was discovered. Because the digital pipeline updates every clock cycle, a slow-moving analog wave might only change by 1 LSB (Least Significant Bit) per tick. If the hardware attempts to scale or divide this difference using right-shift logic (>>), integer truncation forces the slope to 0, completely destroying the derivative data. To resolve this, the hardware utilizes a left-shift accumulator (<< 6) which acts as a pure **Digital Amplifier**, multiplying the microscopic slope differences by a gain of 64 to fully utilize the 8-bit dynamic range of the output DAC.

1.1.2 Real-Time Applications

- **PID Controllers:** Forms the foundation of the "Derivative (D)" term, allowing robotic servos to calculate trajectory and dampen oscillations.
- **FM Demodulation:** Extracts frequency-modulated audio by analyzing the rate of phase change in digital radio receivers.
- **Peak & Edge Detection:** Identifies the exact nanosecond an analog sine wave peaks, as the mathematical derivative perfectly crosses zero at the apex.

1.1.3 Complete Synthesizable Verilog RTL Description

```
‘timescale 1ns / 1ps

module differentiator (
    input  wire        clk,        // System clock
    input  wire        rst_n,      // Active-low reset
    input  wire [7:0]  data_in,    // 8-bit ADC input (analog waveform)

    output reg  [7:0]  data_out    // 8-bit DAC output (the derivative)
);

    reg [7:0] data_prev; // Pipeline register for x[n-1]

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            data_prev <= 8'd128;   // Default to mid-rail
            data_out  <= 8'd128;   // Default derivative of 0 (flat slope)
        end else begin
            // Shift the pipeline
            data_prev <= data_in;

            // Execute First-Order Difference Equation with DC-Bias and 64x Gain
            if (data_in >= data_prev) begin
                data_out <= 8'd128 + ((data_in - data_prev) << 6);
            end else begin
                data_out <= 8'd128 - ((data_prev - data_in) << 6);
            end
        end
    end

endmodule
```

Chapter 2

Verification, Co-Simulation & Waveform Analysis

2.1 IP Block 7: Digital Differentiator Verification

2.1.1 Digital Simulation (Icarus Verilog & GTKWave)

To verify the core arithmetic pipeline independently of mixed-signal translation artifacts, the Verilog module was evaluated using a synthetic digital testbench. A secondary Verilog block (`triangle_gen.v`) was authored to generate a continuous, idealized 8-bit triangle wave to act as the stimulus vector.

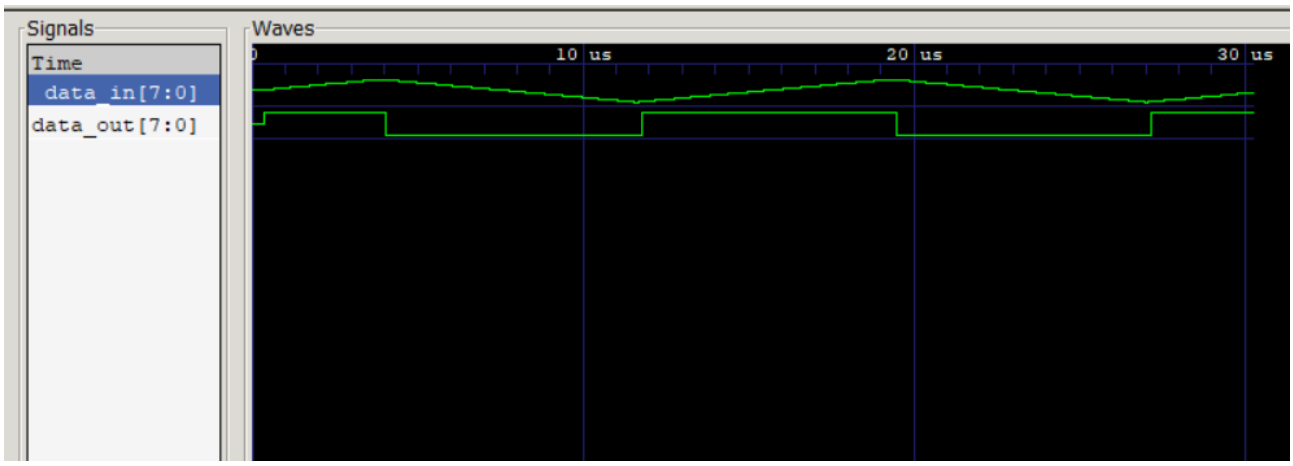

Figure 2.1: GTKWave Simulation demonstrating the instantaneous calculation of the first-order difference equation.

The pure digital simulation successfully proved the fundamental theorem of calculus: the derivative of a constant linear slope (triangle wave) is a constant static magnitude (square wave).

2.1.2 System Integration (eSim Mixed-Signal Scope)

Following digital verification, the system was subjected to physical NGVeri co-simulation to observe the mathematical phenomena translated into the analog continuous-time domain.

Schematic Architecture

To circumvent the limitations of utilizing 8 independent analog voltage sources to construct a binary wave, the purely digital `triangle_gen` IP block was utilized in the schematic to drive the `differentiator` input bus directly. The resulting 8-bit discrete mathematical output was routed

through a `dac_bridge_8` to convert the digital byte array back into a plottable continuous analog voltage.

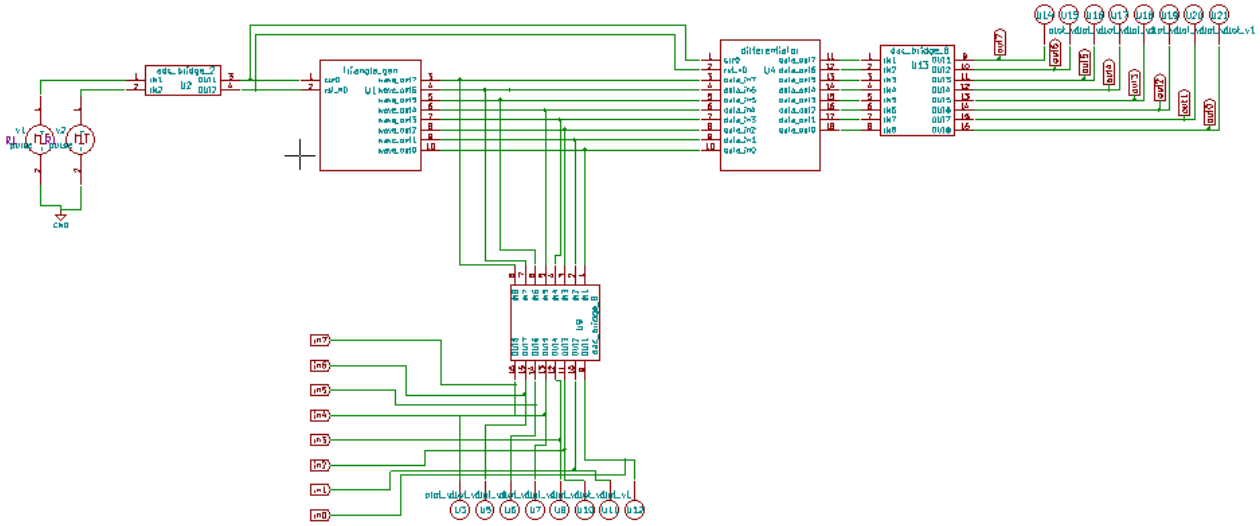

Figure 2.2: Mixed-signal schematic emphasizing the digital generation and analog extraction pathways.

Ngspice Reconstructive Plotting

To visualize the 8-bit bus as a single cohesive analog wave, the Ngspice `.control` script was configured to apply binary-weighted summation across the parallel pins.

```
* Reconstruct the 8-bit Input Triangle Wave (Scaled 0 to 255)
let raw_in = (v(in7)/5)*128 + ... + (v(in0)/5)*1

* Reconstruct the 8-bit Output Derivative Square Wave
let raw_out = (v(out7)/5)*128 + ... + (v(out0)/5)*1

plot raw_in raw_out
```

Interpretation of Output Waveforms

The resulting transient analysis explicitly verifies the success of the digital gain scaling and the real-time execution of the calculus engine.

As shown in Figure 2.3:

1. **Positive Derivation:** When the input wave (green trace) maintains a constant positive slope, the digital hardware correctly evaluates the difference as positive, driving the output DAC (red trace) to a constant high plateau (approx value 192).

2. **Negative Derivation:** The instant the triangle wave reaches its apex and transitions to a negative slope, the Verilog difference equation perfectly crosses the 128 DC-bias baseline and drops to a constant low plateau (approx value 64), mathematically proving the derivation of the input vector.

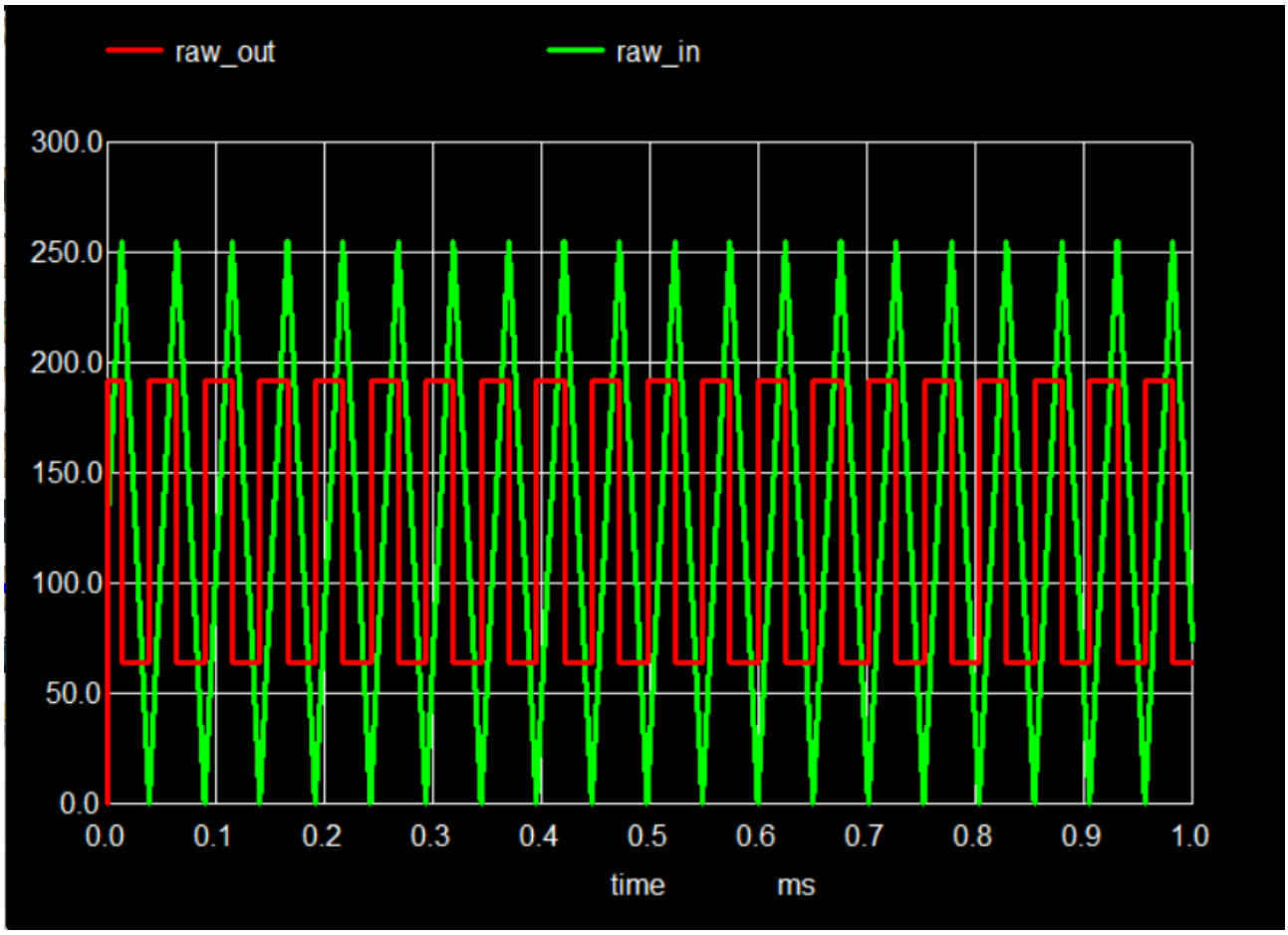

Figure 2.3: Ngspice Transient Analysis proving the hardware-level computation of the derivative.